We can write a formula for the running time: if `|V'|=n`, `|E'|=m`, then

```
T(n,m) = n * T(n-1, m) + m * T(n, m-1);
T(0,0) = 1
T(0,*)=1
T(*,0)=1
```

For a recurrence relation[12] of the form `T(n) = n * T(n-1)` the solution depends on the value of the base term: if, like in this case, `T(0) = 1`, then `T(n) = n!` – this, at least, when we have a single variable; our specific case, where we have two variables, grows even faster…

The last thing we want in a formula for an algorithm's running time is a factorial: a function that grows faster than the exponential function. This means that the algorithm, as it is presented in listing 15.2, can only be used for small graphs.

When you see a factorial pop up in your calculations, it will likely mean that you are computing the same thing over and over again, multiple times.

This is the case for our algorithm as well: let's see an example, with a small graph with just three vertices: let G= ({1,2,3}, E) be our initial graph – for the sake of our example we won't bother about its edges, but focus on the recursion on vertices.

The `for` loop at line #4 of listing 15.2 will issue three calls[13], for the following graphs[14]: